

Backlog API

um sistema onde se possam cadastrar livros ou jogos que o usuário possua mais ainda não leu ou jogou.

A API então retorna para o usuário os itens cadastrados ou uma sublista com itens que combinem com tags e tipos escolhidos. Uma vez que o usuário tenha concluído um item de sua backlog ele o marca como concluído e o sistema pode deleta-lo.

UML

Essa é o digrama de classes UML gerado por código Mermaid. a Imagem gerada está aqui no github

```
---
title: Backlog API
---
classDiagram
    BacklogItem <|-- Jogo
    BacklogItem <|-- Livros
    BacklogRepo -- BacklogItem
    BacklogController -- BacklogRepo
    class BacklogItem{
        +nome : String
        +descricao: String
        +tipo: String
        +tags: List<String>
        +status: boolean
        -BacklogItem()
        -setNome()
        -getNome()
        -setTipo()
        -getTipo()
        -setDescricao()
        -getDescricao()
        -getTags()
        -addTags()
    }
```

```

        -removeTag()
        -getStatus()
        -changeStatus()
        -setSingletag()
        -setTags()
    }
    class Jogo{
        +plataforma: String
        +desenvolvedora: String
        +anoLancamento: String
        -Jogo()
        -setPlataforma()
        -getPlataforma()
        -setDesenvolvedora()
        -getDesenvolvedora()
        -setAnoLancamento()
        -getAnoLancamento()
    }
    class Livros{
        +autor: String
        +editora: String
        +lingua: String
        +anoLancamento: String
        -Livros()
        -getAutor()
        -setAutor()
        -getEditora()
        -setEditora()
        -getLingua()
        -setLingua()
        -getAnoLancamento()
        -setAnoLancamento()
    }
    class BacklogRepo{
        +backlog: List<BacklogItem>
        -getAllBacklog()
        -addBacklogItem()
        -delearPorNome()
    }

```

```

-getBacklog()
-getBacklogConcluido()
-mudarStatusDoItem()
-removerBacklogConcluidos()
-getBacklogPorTipo()
-geBacklogPorTag()
-geBacklogPorTipoETag()
-getNome()
-addTag()
-findAndRemoveltem()
}
class BacklogController{
    +backlogRepo: BacklogRepo
    -addJogo()
    -addLivro()
    -getAllBacklog()
    -getBacklog()
    -getBacklogConcluido()
    -getBacklogJogo()
    -getBacklogLivro()
    -getBacklogTags()
    -getBacklogJogoETags()
    -getBacklogLivroETags()
    -mudarStatus()
    -deletarConcluidos()
    -deletarPorNome()
    -adicionarTag()
}

```

Tabela de Endpoints

Méodo HTTP	EndPoint	Descrição	Status da Resposta
POST	/adicionar-jogo	Adiciona um Jogo ao Backlog	201 Ceated, ou erro implícito

POST	/adicionar-livro	Adiciona um Livro ao Backlog	201 Created ou erro implícito
GET	/todos	Lista todos os itens da Backlog	200 Ok ou 204 NO_CONTENT
GET	/a-fazer	Lista todos os itens não concluídos da Backlog (status == false)	200 Ok ou 204 NO_CONTENT
GET	/concluídos	Lista todos os itens concluídos da Backlog (status == true)	200 Ok ou 204 NO_CONTENT
GET	/Jogos	Lista todos os jogos da Backlog (tipo == Jogo)	200 Ok ou 204 NO_CONTENT
GET	/Livros	Lista todos os jogos da Backlog (tipo == Jogo)	200 Ok ou 204 NO_CONTENT
GET	/tags	Lista todos os itens que compartilhem uma mesma sublista de tags informadas	200 Ok ou 204 NO_CONTENT
GET	/jogos/tags	Lista todos os Jogos que compartilhem uma mesma sublista de tags informadas	200 Ok ou 204 NO_CONTENT
GET	/livros/tags	Lista todos os Jogos que compartilhem uma mesma sublista de tags informadas	200 Ok ou 204 NO_CONTENT
PATCH	/alterar-status	altera o status de um item da Backlog	200 OK ou 404 NOT_FOUND
PATCH	/adicionar-tag	adiciona uma nova tag a um item previamente cadastrado passando seu nome	200 OK ou 404 NOT_FOUND
DELETE	/deletar	Deleta um item da Backlog ao informar seu nome	200 OK ou 404 NOT_FOUND
DELETE	/deletar-concluídos	Deleta todos os itens da backlog marcados como concluídos (status == true)	200 OK ou 404 NOT_FOUND